



Soccer Infrared - Team Hertzquake

Elettra Robotics Lab (ERL) & IIS L. Cobianchi, Verbania

ITALY

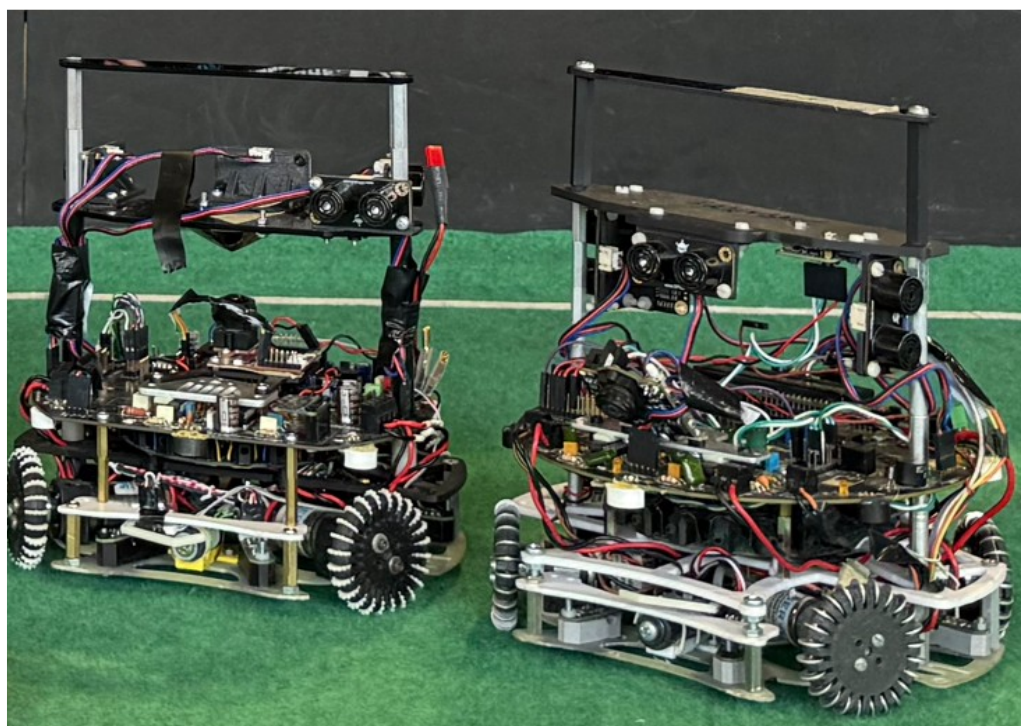
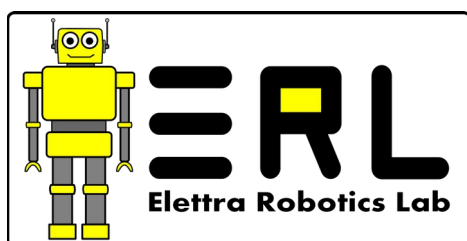


Table of Contents

1. Introduction and robot's initial overview

1.1. *I/O Pinout of **Viper S³** and **Phantom** main board*

2. Robot's general logic and behavior

2.1. *Attack and defense strategies*

2.2. *Attack maneuvers*

3. Electronic hardware

3.1. *Custom PCBs*

3.1.1. Kicker Board

3.1.2. Main Board

3.2. *Sensors and measuring units*

3.2.1. IMU

3.2.2. Pixy 2 Camera

3.2.3. Ultrasonic sensors

3.2.4. Custom line sensors

3.2.5. Custom BallSeeker Board

4. Mechanical and structural hardware

4.1. *Proprietary omnidirectional wheels*

4.2. *Chassis*

4.3. *Dribbler and Kicker*

4.4. *Mirror*

4.5. *Robots Design*

4.6. *Omnidirectional movement*

5. Software

5.1. *Camera*

5.2. Main MCU

5.2.1. Communications management

5.2.2. Ball following algorithm

5.3. Secondary MCU

6. Gallery

Team members

- Zani Dimitri
- Todeschini Alessio
- Atifi Wassim
- Pittoni Ariele

Mentors

- Raimondo Sgro'
- Sergio Villa

Team Introduction

We are the Italian soccer Infrared team **Hertzquake**. Our team consists of two students in their 4th year of high school and two students in their 5th at the **Lorenzo Cobiانchi** Institute, in **Verbania**, one following the *electronics systems design* course and other three members following the *ICT course*. Thanks to this experience the whole school got closer to the whole world of robotics and to its competitions.

Our team has been assisted by professor Raimond Sgro' whose insight and experience with the past competitions helped us get prepared for **Vienna**.

Our team is organized as follows:

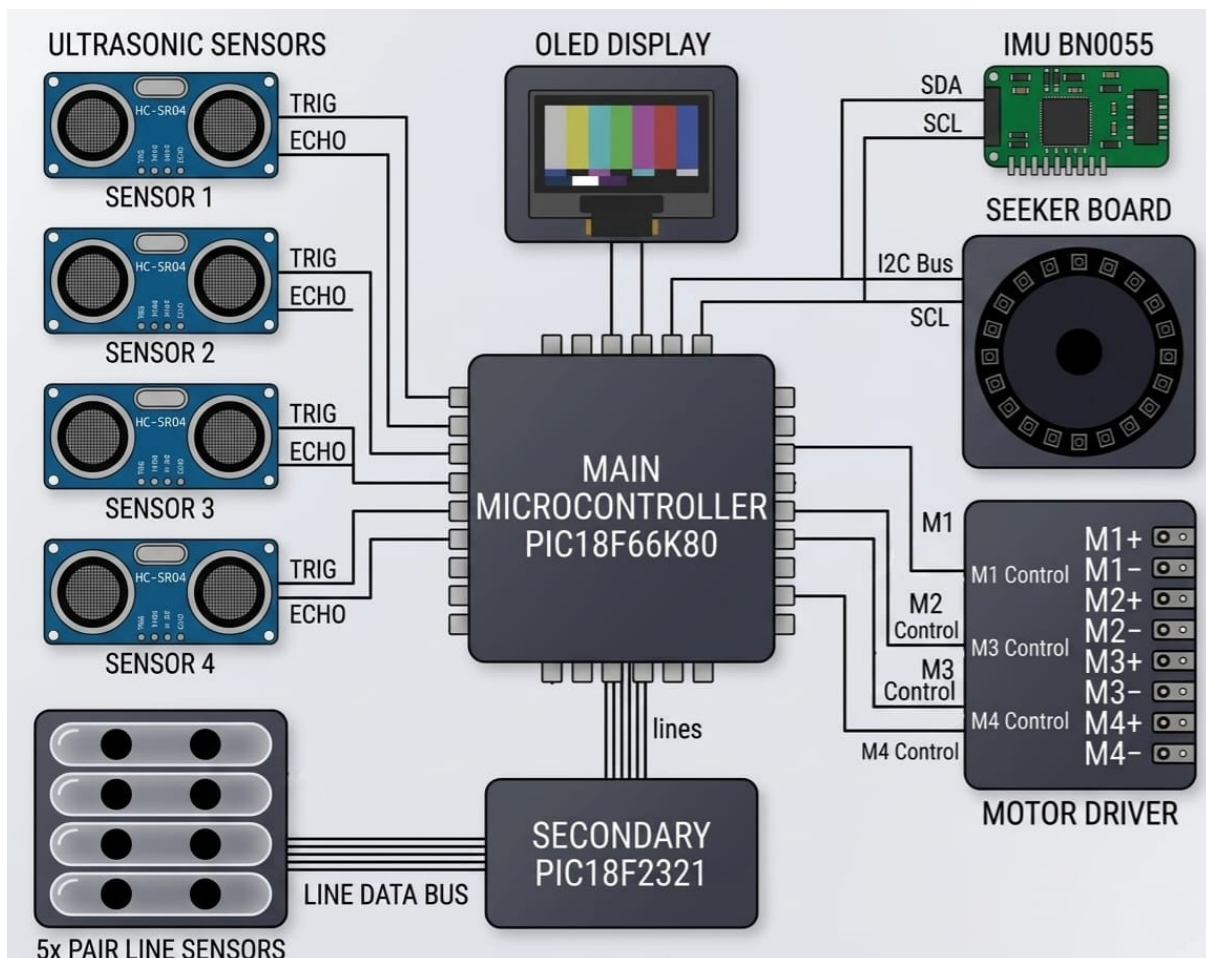
- Todeschini Alessio : **Main MCU software design**. attack strategies

-
- Zani Dimitri : **CAD Design**. Overall robot structure, omni wheel design and production
 - Atifi Wassim : **Electronics Design**. Custom PCB design, dribbler design, production, documentation and poster.
 - Pittoni Ariele : **Secondary MCU software design**. defense strategies, sensors management, Camera vision.

1. Introduction and robot's initial overview

This year our robots are **Viper S³** and **Phantom**. We participated with them in the **Robocup Junior Academy 2026, Catania**. **Phantom** is the main attacker while **S³** is the "goalkeeper", but the roles in the game are dynamic and change depending on the situation.

Structurally speaking, they have only one difference: the main PCB. The boards used aren't the same on robots, this is not an obstacle because we can use the same version of code for each robot. All of the **Viper S³** and **Phantom** components communicate with each other via different protocols. Here follows a block diagram of the robot's structure.



Viper S³ and Phantom Block Diagram



Our robots follow a hierarchical structure where the top is embodied by the main MCU: a **PIC18F66K80**. This 8 bit MCU operating at *48 MHz* and receives data using multiple UART ports and direct GPIO connections. The second important MCU on the main logic board is the **PIC18F4525** (or **PIC18F2321** on Phantom), whose objective is to read the analog custom made line sensors and inform the central unit when a line is crossed through a *Low Priority Interrupt* and then signals which sensor has been activated using one of the 5 direct lines. Additionally the robots use a removable Bluetooth module which is used in the robots strategies and for remote debugging purposes using a computer.

Other peripherals that the logic board manages are :

- Four **DFRobot SEN0307**
- One **Ball touch trigger**
- One **Oled SSD1306 Display**
- One **BNO055 IMU**
- One **Kicker control board**
- One **Custom Kicker**
- One **Pixie**
- One **Custom BallSeeker**

The main logic board is also equipped with numerous LEDs and a selector used for debugging purposes and tactics switching during matches.

1.1. I/O Pinout of Viper S³ and Phantom main board

PIN	I/O	Function	PIN	I/O	Function
RA0	AI	ULTR_DX	RA6	-	-
RA1	AI	ULTR_SX	RA7	-	-
RB0	DI	INTLIN (Interrupt Line)	RD0	DO	M1_A
RB1	DI	LIN_AV_DX	RD1	DO	M1_B

RB2	DI	P_START (SW1)	RD2	DO	M2_A
RB3	DI	AD_SEL	RD3	DO	M2_B
RB4	DI	CAMERA	RD4	DO	M3_A
RB5	-	-	RD5	DO	M3_B
RB6	DI	LIN_AV	RD6	DO	M4_A
RB7	DI	LIN_IND	RD7	DO	M4_B
RC0	DO	ROBOCUP MODULE	RE0	DO	LED_ON (LED1)
RC1	DO	SHOOT	RE1	DO	LED_PWR (LED2)
RC2	-	-	RE2	DO	LED_UL (LED3)
RC3	-	I2C_SCL	RE3	-	-
RC4	-	I2C_SDA	RE4	DI	P_BUS (SW2)
RC5	DO	BUZZ	RE5	DI	P_AUX
RC6	-	-	RE6	-	-
RC7	-	-	RE7	-	-
RF0	DI	IR0	RG0	-	-
RF1	DI	IR0DX	RG1	DI	LIN_DX
RF2	DI	IR0DX2	RG2	DI	LIN_SX
RF3	DI	IR0_REAR	RG3	-	-
RF4	DI	IR0SX2	RG4	DI	TOUCHPIN (Ball Presence)
RF5	DI	IR0SX	RG5	-	-
RF6	DI	RUN	RG6	-	-
RF7	DI	SEL_AUX	RG7	-	-

2. Robot's general logic and behavior

As we said before the two robots have different roles on the field. While developing their behavior we pictured in our minds the three main different scenarios that each robot could encounter, then we implemented each robot's behaviour according to their role and to the probable scenarios they might encounter.

2.1 Attack and defense strategies

By leveraging their built-in Bluetooth modules, we designed the robots to constantly exchange critical state information. With **S³** designated as the defender and **Phantom** as the striker, the first key piece of data shared is the "*I'm Active(K)*" signal. If a robot is penalized for going out of bounds and removed from the field for a minute, this signal drops. This interruption triggers a safety protocol: if the remaining robot cannot see the ball, it will immediately fall back to defend the net, regardless of its primary role. Otherwise, when the robot is in close proximity to the ball, it will act like an attacker, taking advantage of its position. This straightforward logic significantly reduces conceded goals by ensuring our side is rarely left unguarded.

Additionally, the robots transmit their "*Ball status*," indicating whether they can currently see or possess the ball. Based on this data, the robot actively engaging the ball will push forward to score, while its teammate automatically drops back to a defensive position.

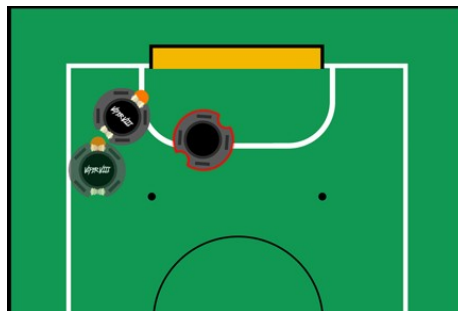
Finally, if both robots lose sight of the ball, their actions are dictated by the **A/D** (*Attack/Defense*) switch. The designated defender retreats to guard the allied goal, whereas the attacker navigates toward the midfield to

maximize its chances of locating and chasing the ball. These precise movements are executed using ToF (Time of Flight) ultrasonic distance sensors, coupled with camera data that helps align the defending robot perfectly with our goal. Furthermore, the **A/D** selector dictates the specific role broadcasted to the partner robot.

2.2 Attack maneuvers

Here are a few scoring maneuvers we have programmed into the robots:

- **Standard / Torsion Shot:** Upon confirmation from the touch sensor that the ball is securely gripped by the jaw, the robot checks its camera feed. If the enemy goal is currently visible, it fires the kicker immediately. If the goal isn't in sight, the robot initiates a rotational sweep. By relying on compass data and ultrasonic sensors for precise orientation, it turns until the target comes into view, triggering the shot the exact moment the goal is detected.
- **Catch and Retreat (Fall Back):** When the ball is trapped in one of the opponent's corners, the robot will drive it against the wall to force a rebound. Once the ball rolls back out, the robot reverses to a safe position that allows for a clear, unobstructed line of sight to the target goal. *(Note: Below is the code snippet corresponding to this maneuver).*



```

PORT:
//*****TRASMISSIONE*****
if(IRO == 1 || IRODX == 1 || IROSX == 1 || TOCCO == 1){ //Garato se ho la palla vicino e davanti
    // BOOP();
    TXREG2 = 'K';
}
//commento altrimenti non funziona debug
//*****RICEZIONE*****
if (PIR3bits.RC2IF==1) { //controlla la ricezione
    DATO = RCREG2;
    RCREG2 = 0;

    if (DATO== 'K') { //aggiunge un 1 nel bit piú significativo del ASCII, k, K

        FLAG_BT = 1;
        DATO = 0;
        PIR1bits.RC1F=0;
    }

    if (RCSTA2bits.OERR ==1||RCSTA2bits.FERR ==1) { //verifico gli overflow e resetto
        RCSTA2bits.CREN=0;
        RCSTA2bits.OERR =0;
        RCSTA2bits.FERR =0;
        RCSTA2bits.CREN=1;
    }

}

if(contBT <= 100 && FLAG_BT == 1 && (AD_SEL == 1 || (AD_SEL == 0 && IRO == 0 && IRODX == 0 && IROSX == 0 && TOCCO == 0))){
    LED_UL=1;
    VEL=0.7;
    K_BUS_H = 2;
    VEL = VEL - 0.2;
    if (ULREAR <= 50) {calcolaPwmDi=(avanti,ERR);}
    else{calcolaPwmDi=(indietro,ERR);} //accentro avanti e indietro

    if (ULDX < SOGLIA_CENTRO && ULSX > SOGLIA_CAMPO) {
        calcolaPwmDi=(sx,ERR);
    }
    if (ULSX < SOGLIA_CENTRO && ULDX > SOGLIA_CAMPO) {
        calcolaPwmDi=(dx,ERR);
    }
    BOOP();

    contBT++;
goto CICLO;
}
else{
contBT = 0;
FLAG_BT = 0;
LED_UL=0;
};
}

```

"Kick on the line " maneuver's code extract

3. Electronic hardware

The electronic hardware on our robots allows us to perform a wide variety of tasks . Without these external reference points it would be nearly impossible for our robots to move in straight lines, follow the ball, decide when to shoot and when to stop before it gets out of bounds. On top of that our custom PCBs house all the microcontrollers and other hardware used by the robot to perform its actions and by us to monitor the state of each algorithm.

3.1. Custom PCBs

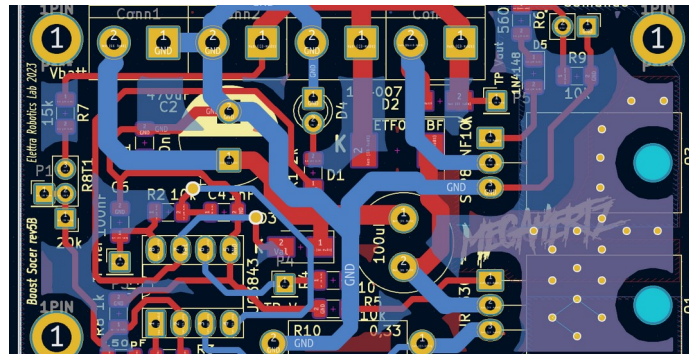
Our custom PCBs were all designed using **Kicad 8**, an open source and free EDA software perfect for both beginners and more skilled designers. Through its schematic and PCB editor we were able to draw our boards from the ground up seamlessly . We found plenty of plugins that aided our development process. Once the boards were ready for production we partnered with a local company named **Millennium Dataware SRL** who offered to produce them and sponsor us. Although we had a stable supplier for our PCB we still had to rely on some 3rd party services and their fast shipping for out of schedule batches. The first version of the kicker board used to generate a lot of electrical noise and its capacitor charging speed was low, due to a design flaw in the placement of the components. With those mistakes we were able to improve the design and make better ones.

3.1.1 Kicker Board

The **kicker boost board**, also referred to as “**Shot board**”, is designed to actuate a “homemade” electromagnetic solenoid. The “**Shot board**” purpose can be split into two main functions:

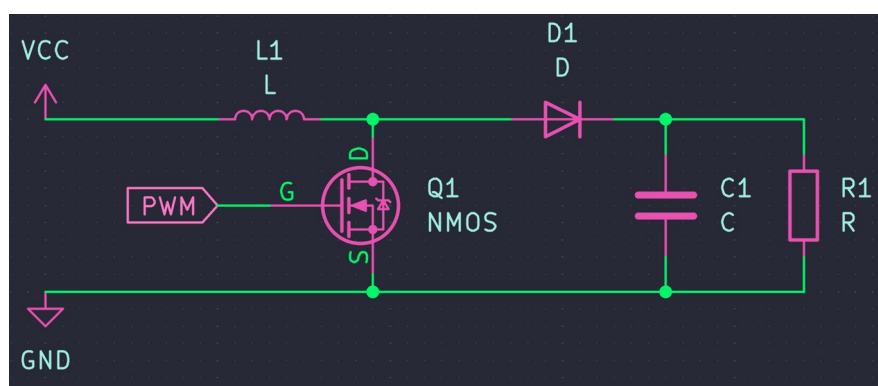
1. Charge the capacitors at a voltage of 48 V using a step-up circuit and a supply of $\sim 12\text{ V}$ (3S lipo battery voltage).
2. Quickly discharge the capacitors in the solenoid when the main MCU requests it through a direct line. This releases a high-current burst through the copper windings which generates a strong magnetic field that accelerates the iron piston inside the solenoid.

The board design is based on the UC3843 switching regulator.



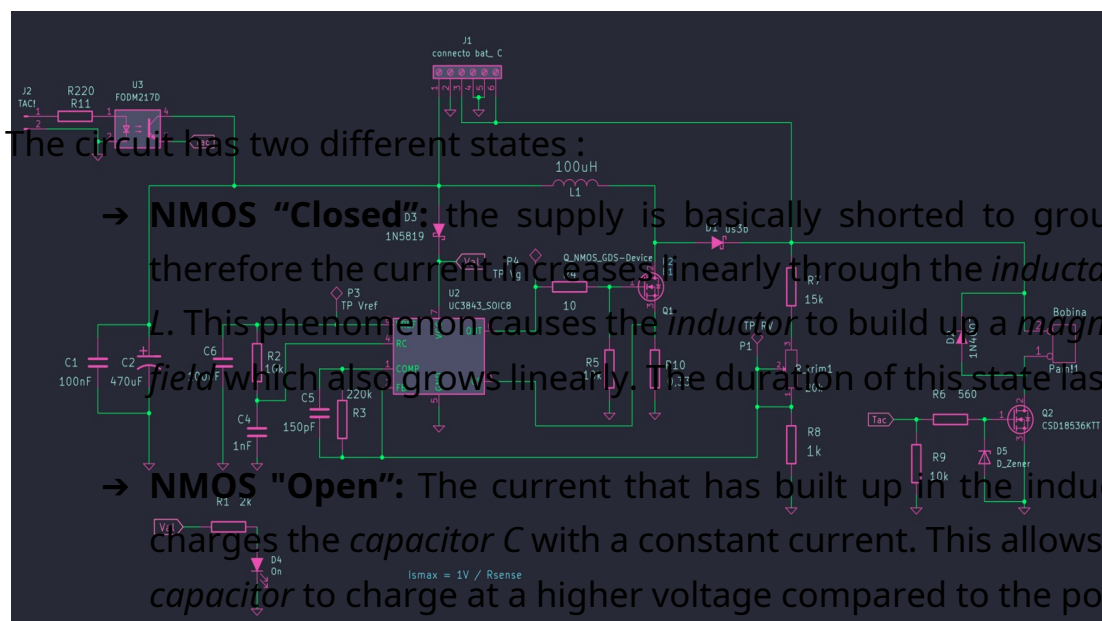
Shot card V5 board's PCB

From the schematic diagram it is possible to spot the DC-DC boost



converter circuit. For clarity sake here is a “bare-bone” version of the board.

Kicker module demonstrating circuit



supply one. The diode D controls the current flow to prevent a back current to flow in the supply. The duration of this state last **T2**

Equation of Step-Up circuit: $V_o = V_i \cdot \left(1 + \frac{T_2}{T_1}\right)$

Regulating **T1** and **T2** it is possible to regulate the output voltage. The input signal of the circuit is given by a **PWM** generated signal, with variable **duty cycle**. The switching regulator is the UC3843, in the smd package.

Using the following equations we were able to calculate the output velocity of the ball should end up with, ideally.

E = energy / C = Capacitance / V = Voltage

$$E = \frac{1}{2} \cdot C \cdot V^2$$

Ideally the conversion of energy from potential energy, in the capacitor, into kinetic energy in the ball should be perfect, thus the velocity of the ball should be defined as:

$$V = \sqrt{\frac{2E}{C}}$$

Since the rules limit the kicker voltage at 48 V we had to increase the capacitance of the overall system, that's why we've increased the capacitors to 15000 μF .

3.1.2. Main Board

The Main board is composed of two main parts:

- The Logic board
- The Power board

The logic board is the heart of our robots. It houses most of the vital components of the project. Here we can find both Microchip© **MCUs** . The primary controller runs at a frequency of **48 MHz** while the secondary one runs at **4 MHz**. The board also hosts serial ports for the **OpenMV H7** and **Bluetooth** boards. Last but not least there are two **I2C** channels. One hosted by the **secondary MCU** whose purpose is to display, on an **OLED**, the state of each line sensor, as well as calibration uses. While the other one has as its master the **main microcontroller**. This channel is populated by another **OLED Display** and by the **BNO055 IMU**. Lastly each **MCU** has an **ICSP** connector and switch, used for programming.

If we sum the number of pins available on the board, coming from both **MCUs**, the total amounts to almost 100 GPIO, fully populated. This allows us to have a total control of all the onboard hardware.

The board is equipped with some additional circuitry such as the ultrasonic sensors attachments, debug and strategies selectors, a buzzer and line sensors state LEDs (that also have the function of communicating the state of each line sensor from the secondary MCU to the main one through a direct connection). The board is also equipped with 6 infrared sensors that allow us to know when the ball is close to the robot.

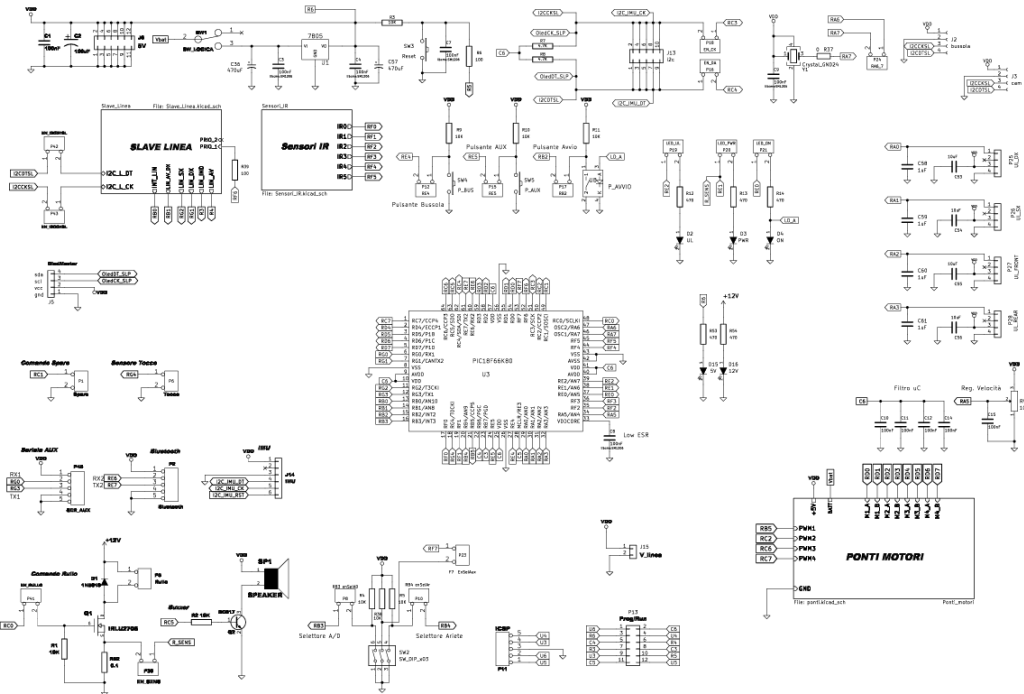
The main purpose of the Power board is to control each of the four motors used for omnidirectional movement. The board presents two **ST L6207N H bridges**. Each IC can control the direction and speed of two motors. They require some external circuitry to work properly and to protect both the IC and the MCU in case of short circuit or overheat.

Lastly the board hosts two 5 V regulators . For the regulator we opted for switching ones. We calculated the power loss on the old 5 V regulator (**L7805**) when supplied with 12 V. Knowing that our logic circuitry draws around 200 mA it's easy to find the power dissipated by the IC:

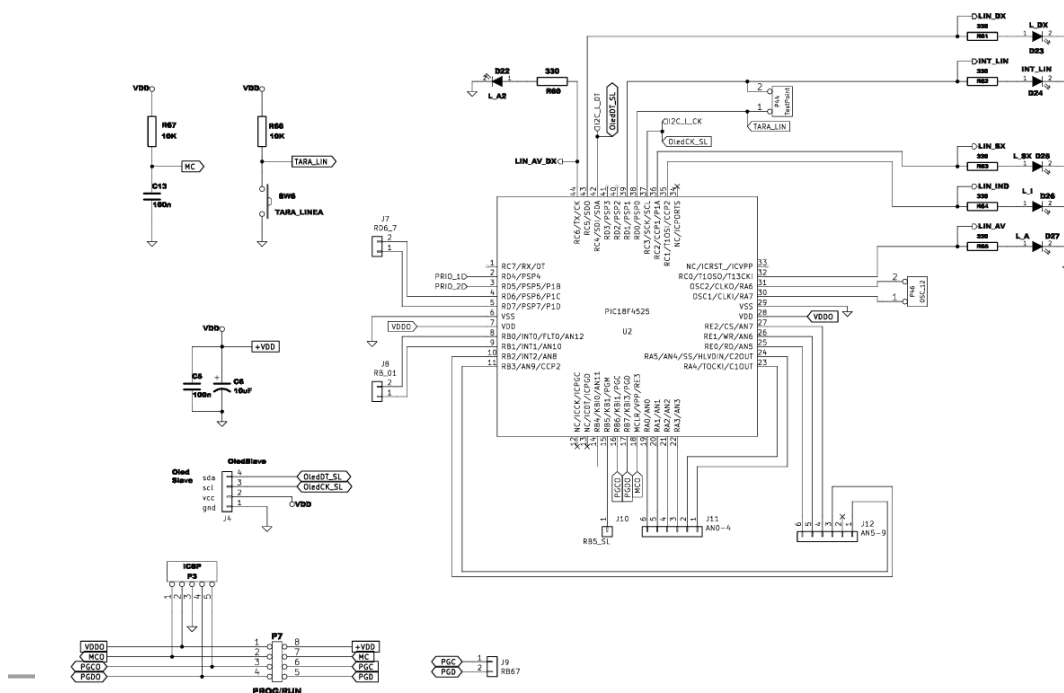
$$PD = (V_{CC} - V_{regulator}) \cdot I_{logic} = (12 - 5) \cdot 200 \cdot 10^{-3} = 1.4 \text{ W}$$

Which is a lot. Not to mention that there is a power loss of almost 60% and using this IC means that it would be mandatory to provide it with a beefy heatsink since from the datasheet it specifies that the temperature of the component rises 60° at every watt the component dissipates.

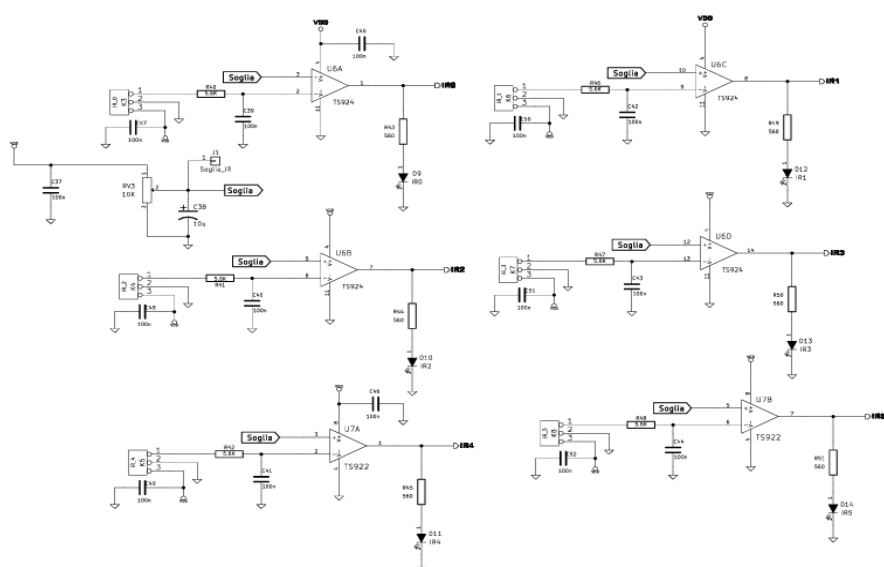
. The overall working principle of a switching IC is the same used in the kicker step-up circuit.

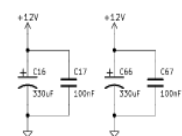


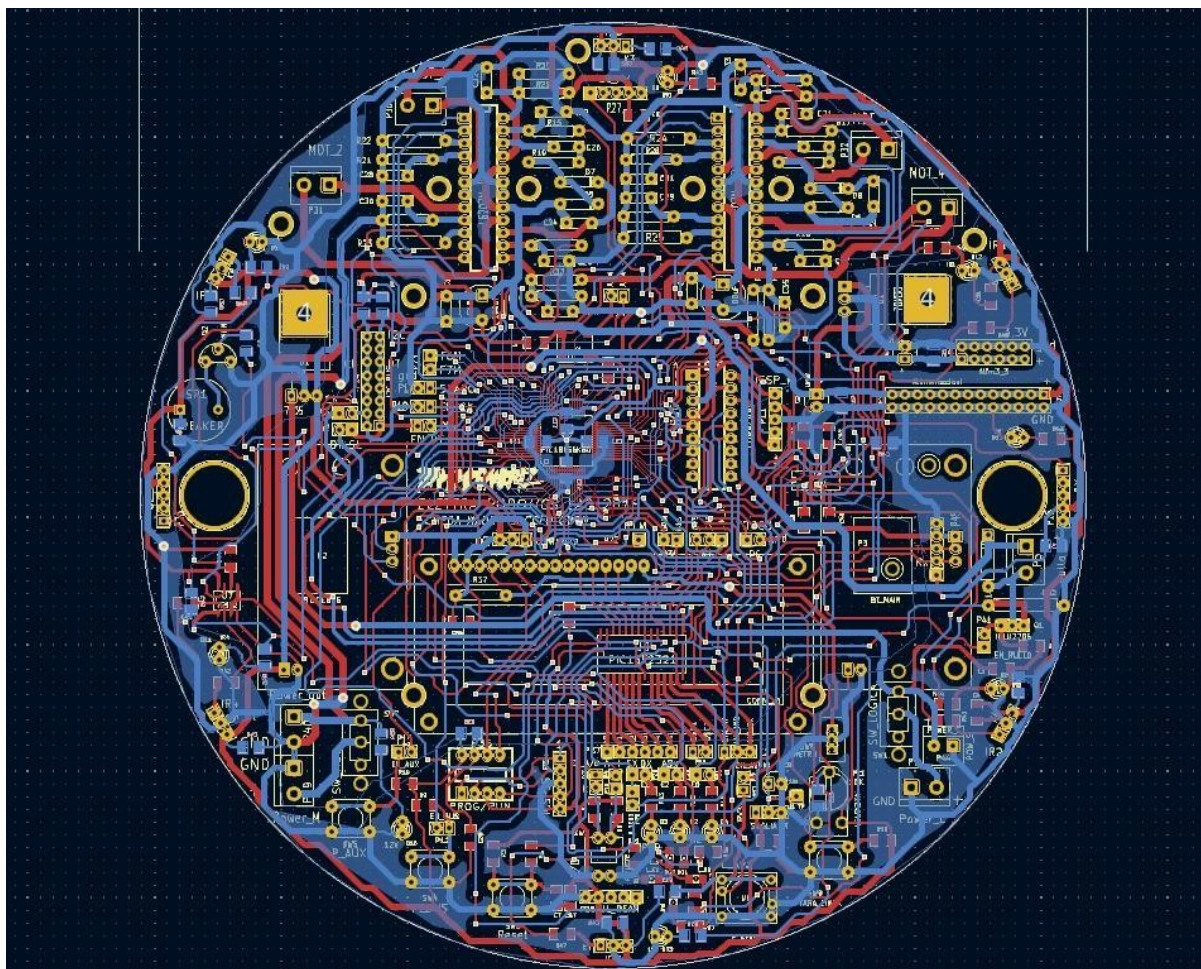
Main MCU S³ Schematic



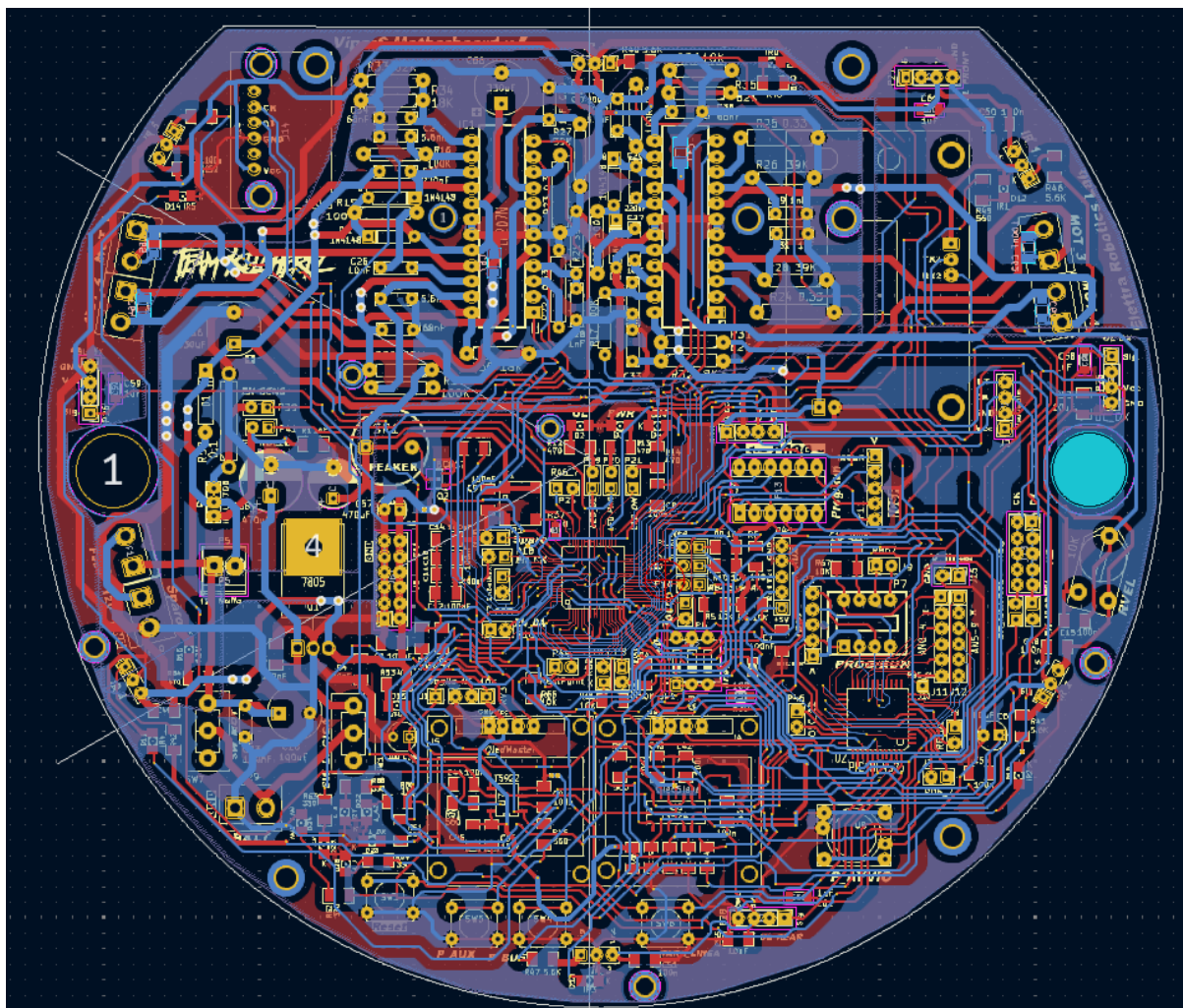
Secondary MCU S³ Schematic







MainBoard Phantom PCB design



MainBoard S³ PCB design

3.2. Sensors and measuring units

Both **Viper S³** and **Phantom** are equipped with a diverse array of sensors distributed across their exteriors. These components are vital for designing and executing maneuvers that demand high-precision movements. The most critical measurement units are outlined below:

3.2.1 IMU

The Inertial Measurement Unit (IMU) is a crucial component for executing and controlling the robots' movements. This device allows us to track motion across 9 degrees of freedom (9-DOF) by combining data from three internal sensors: a gyroscope, an accelerometer, and a magnetometer, each measuring 3 degrees of freedom. Because our robots operate on a flat surface like rovers, not all of these measurements are strictly necessary for our specific application. Nevertheless, transitioning to this IMU was a massive upgrade over the standalone magnetometers used in our older iterations, as it offers faster response times and significantly better resistance to external electromagnetic noise.

Specifically, we utilize the **BNO055** sensor. It calculates the robot's current orientation angle relative to a baseline "north" position, which is calibrated at the start of each cycle by pointing the robot directly at the opponent's goal. The orientation feedback from the IMU is continuously fed into an algorithm featuring a PID controller, designed to correct any rotational drift or misalignment. The corrective output from the PID is then integrated into the PWM (Pulse Width Modulation) equations, ensuring precise and stable control over the robot's drive system.

```
void leggiBusCalcolaErr(void){
    //appBus = BUS;
    int i;

    CMPS03(); // restituisce il dato in BUS 0-360

    if (BUS > (BUSINI+TORX)) {BUS = BUS-(BUSINI+TORX);} //tolgo l'offset iniziale
    //aggiungo torsione
    else { BUS = 360 - (BUSINI+TORX-BUS) ;} //nei due casi

    //calcolo l'errore della bussola
    if (BUS<=180) //errore positivo, rallento a SX, accelero a DX
    {ERR = BUS;}
    else
    {ERR = BUS-360;}; //ricavo l'errore negativo complemento a 255

    if ((ERR<5 && ERR>-5)) iErr = 0; //||DIR!=0//calcolo la parte integrale dell'errore
    else iErr = 0; //IERR = IERR + 0.0005*ERR

    ERR = ERR + iErr; //disattivo momentaneamente l'integrale

    //if (ERR > 180) { ERR -= 360; }
    //if (ERR < -180) { ERR += 360; }
    //debug errore
    /*
    if (i>50) {
        debugDato(ERR);
        i=0;
    }
    else i++;
    */
}
```

IMU Reading Function

```
//Routine di centratura con PID

void CENTRA_PID(void)
{
    int PWM=0;
    int PID=0;
    float pp=0;
    float ii=0;
    float dd=0;
    int lasterr=0;
    float kp=1.5;
    float ki=0.05;
    float kd=6.0;

    CMP803(); // restituisce il dato in BUS 0-255

    if (BUS > BUSINI) { BUS = BUS - BUSINI; } //tolgo l'offset iniziale
    else { BUS = 255 - (BUSINI - BUS); } //nei due casi

    if (BUS > TOLBUS_P && BUS < TOLBUS_N) {
        PORTD = 0;
        goto FINE_BUS_PID; } // se sono nei limiti mi fermo

    if (BUS < 128) //errore positivo, rallento a SX, accelero a DX
    { LOCERR = BUS; }
    else
    { LOCERR = BUS - 255; } //ricavo l'errore negativo complemento a 255
    pp = kp * LOCERR;
    ii = ii + (LOCERR * ki * 0.005);
    dd = (lasterr - LOCERR) * kd * 0.005;
    lasterr = LOCERR;

    PID = pp + ii + dd;

    Delayms(5); //5ms@16MHz
    if (PID < 0)
    { PID = PID * -1; }
    PWM = 150 + PID;
    if (PWM > 1023)
    { PWM = 1023; }
    if (PWM < 0)
    { PWM = 0; }

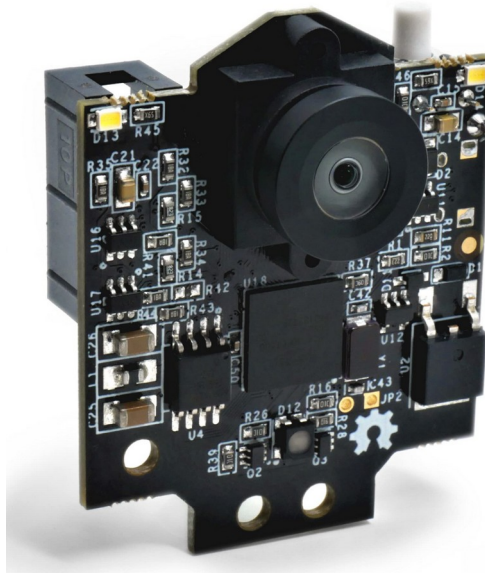
    if (LOCERR < 2 && LOCERR > -2)
    { PORTD = 0; }
    else {
        if (BUS > 128) //se maggiore di 180 vado a DX
        {
            PORTD = DES_PLUS;
            SetDCPWM45(PWM); //su 1024 M1 SX
            SetDCPWM42(PWM); //su 1024 M2 SX
            SetDCPWM43(PWM); //PWM M3 DX
            SetDCPWM44(PWM); //PWM M4 Dx
        }
        else //minore di 180
        {
            PORTD = SIN_PLUS; //se minore di 180 vado a SX
            SetDCPWM45(PWM); //su 1024 M1 SX
            SetDCPWM42(PWM); //su 1024 M2 SX
            SetDCPWM43(PWM); //PWM M3 DX
            SetDCPWM44(PWM); //PWM M4 Dx
        }
    }
}

FINE_BUS_PID:
.
```

PID Calculation Function

3.2.2 Pixy 2 Camera

the **Pixy 2** CMUcam5 Image Sensor. This powerful module guarantees a stable frame rate and features a fast onboard processor that handles all the image recognition algorithms directly. We mounted the camera on the



robot facing forward. The camera communicates with the logic board via a serial port.

Pixy 2 on Phantom

3.2.3 Ultrasonic sensors

Ultrasonic sensors were among the first distance measurement tools integrated into our robots. Thanks to their rapid response times and solid precision, they are ideal for on-the-fly obstacle detection during operation. However, because they output an analog signal, the readings are susceptible to electrical noise, which can compromise data reliability. For

this project, we specifically utilize the waterproof DFRobot SEN0307 ultrasonic sensor.



3.2.4 Custom line sensors

Both **Viper S³** and **Phantom** are equipped with 10 line sensors. The sensors are positioned, in pairs, in a pentagon shape having the right, rear and left sensors form a cross, while the last two pairs are positioned at an angle of 15 and -15 compared to the robot forward center. Each vertex of the pentagon has an internal and external sensor. With this configuration we can firstly identify the presence of the line using the external sensors, then once the robot gets too out of bounds and triggers an internal sensor a reentry maneuver will be performed.

Each line sensor consists of a small PCB with 2 pairs formed by an LED and a phototransistor one next to the other. The light difference between the field and the line will cause the phototransistor to change his voltage value. This analog value is read for each pair by the secondary **MCU** using an analog pin. All the line sensors are mounted on custom made 3D printed support bolted to the bottom frame of the robot.



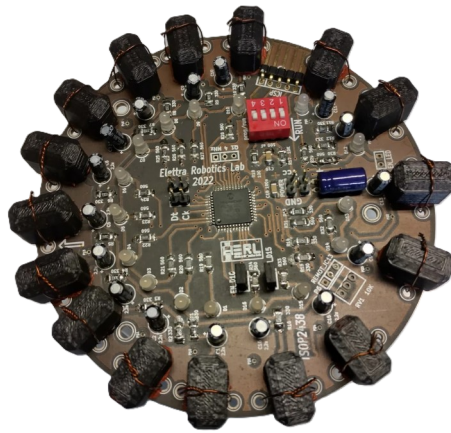
Line sensor board near the 3D printed support

3.2.5 Custom BallSeeker Board

This board is a specialized, 360-degree infrared sensor designed for competitive robotics, such as RoboCup Junior, to track IR-emitting balls without any blind spots. The circular PCB features an outer ring of sixteen TSOP2438 infrared receivers, which are specifically tuned to the 38 kHz frequency emitted by official competition balls.

In the center of the board, a surface-mount microcontroller continuously processes the incoming sensor data. It communicates the ball's relative position to the robot's main controller using the I2C protocol, which is clearly indicated by the SDA and SCL pins. Because it uses I2C, the board includes a prominent four-position red DIP switch, allowing users to easily configure its hardware address so multiple devices can share the same communication bus without interference.

Lastly, the power delivery system is designed for stability. Powered via standard VCC and GND pins, the board utilizes large electrolytic capacitors to prevent voltage drops caused by the robot's motors, alongside smaller ceramic capacitors that filter out high-frequency electrical noise, ensuring reliable and accurate sensor readings.



Custom BallSeeker

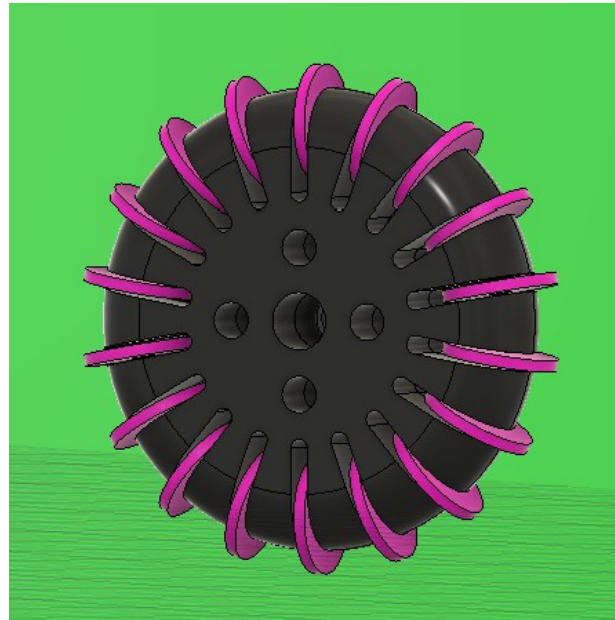
4. Mechanical structure and hardware

The main goal for mechanical improvements this year was to decrease the height of our robots compared to the previous ones, in order to enhance stability and movements. To make this possible we had to reduce the distance between the two main frames, which is not an easy task due to the presence of the kicker's structure. Another task for this year mechanics was to change the angulation of the wheels for a better performance, and also produce a new kind of omni wheel with enhanced grip.

4.1. Custom omnidirectional wheels

Our designers landed on a tooth-shaped washer to greatly improve the performance of our wheels, which will replace the old smooth washers we used. There we concluded that the new wheels help with both the responsiveness of the movements and also with the head-to-head challenges when 2 robots clash into each other.

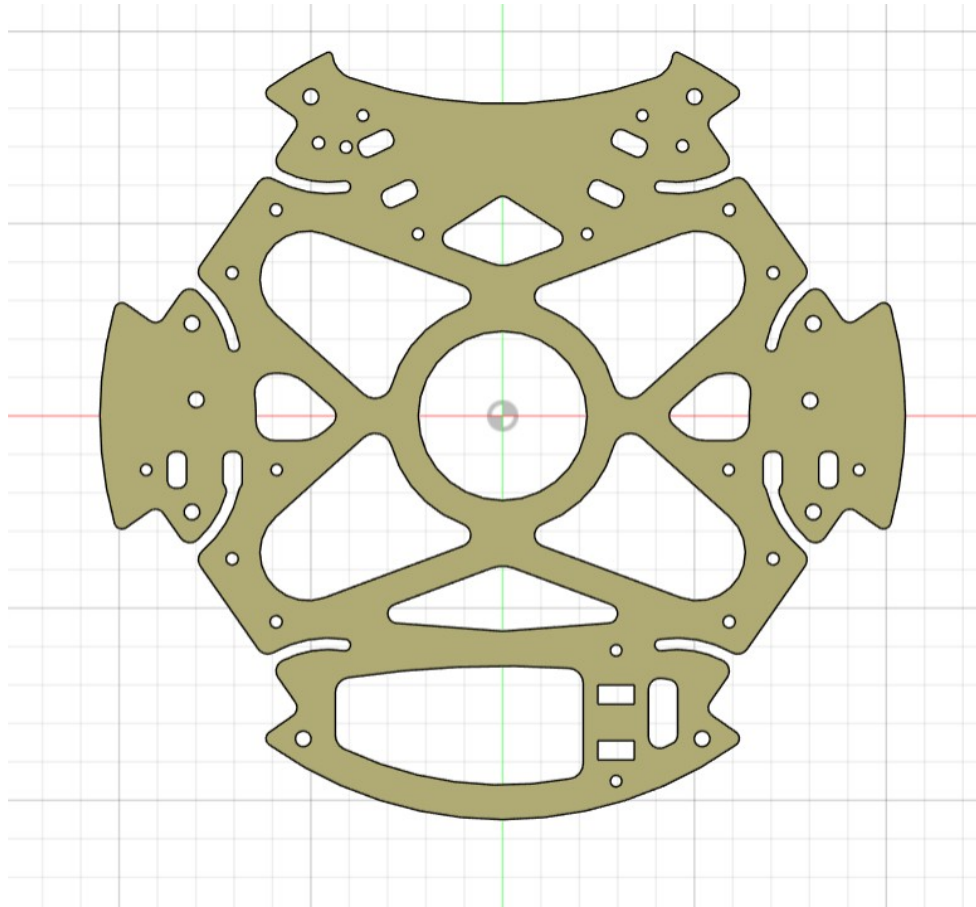
To avoid damaging the field and to make them easier to produce in mass, the washers are 3D printed.



Omnidirectional wheel with smooth washers

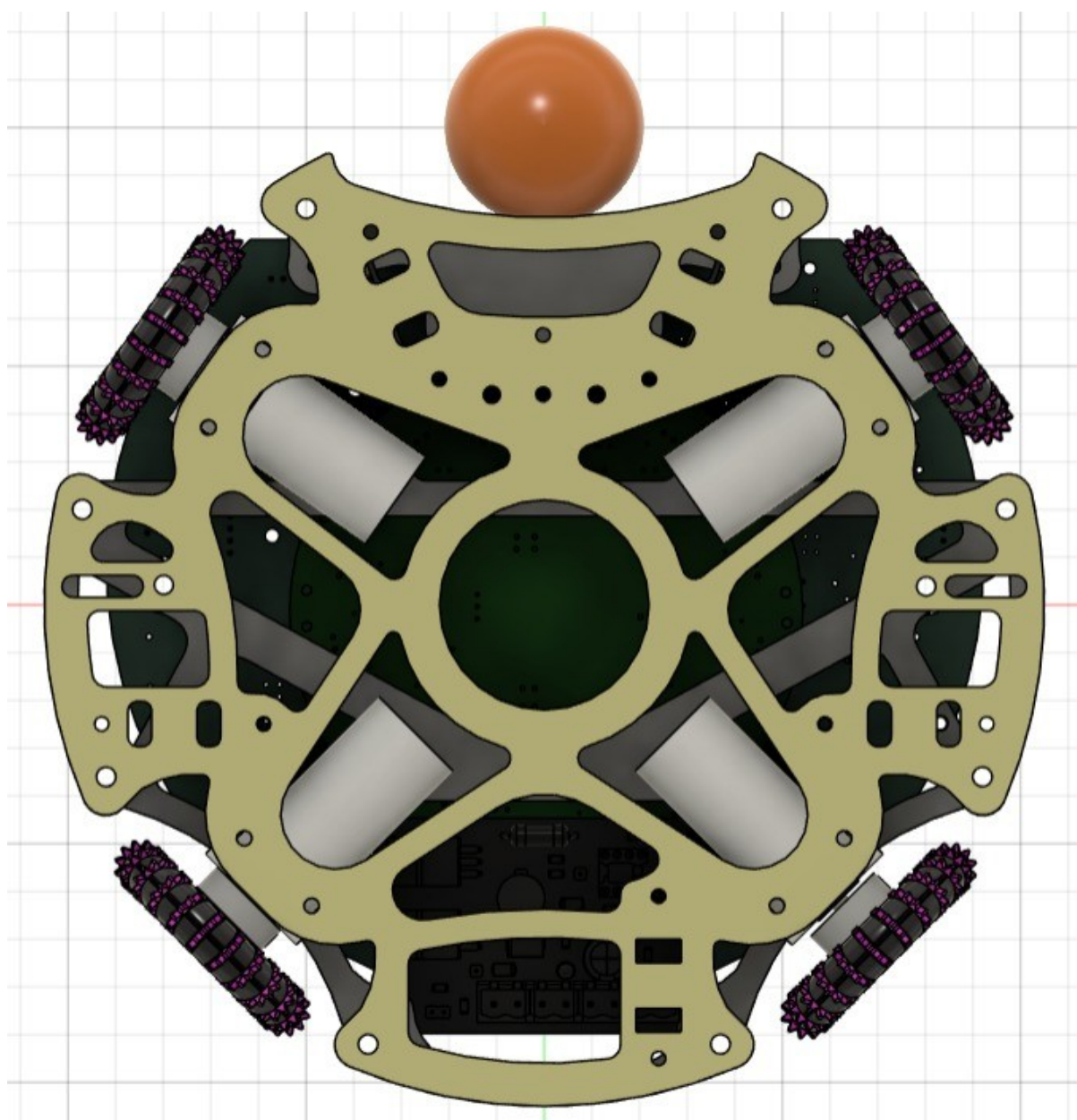
4.2. Chassis

For the **chassis design**, we retained a four-motor configuration set at 110-70-degrees angles to maximize the robot's power output. Additionally, we expanded the wheelbase distance to enhance overall stability. Unlike previous iterations, the **kicker coil** is now mounted on the upper section of the frame, which effectively minimizes any interference with the drivetrain. As for the **line sensors**, they continue to be laid out in a pentagonal pattern, featuring two phototransistors per sensor block. This specific arrangement is engineered to accurately calculate the exact direction in which the robot crosses the white boundary line.



The lower frame design, with curved suspensions

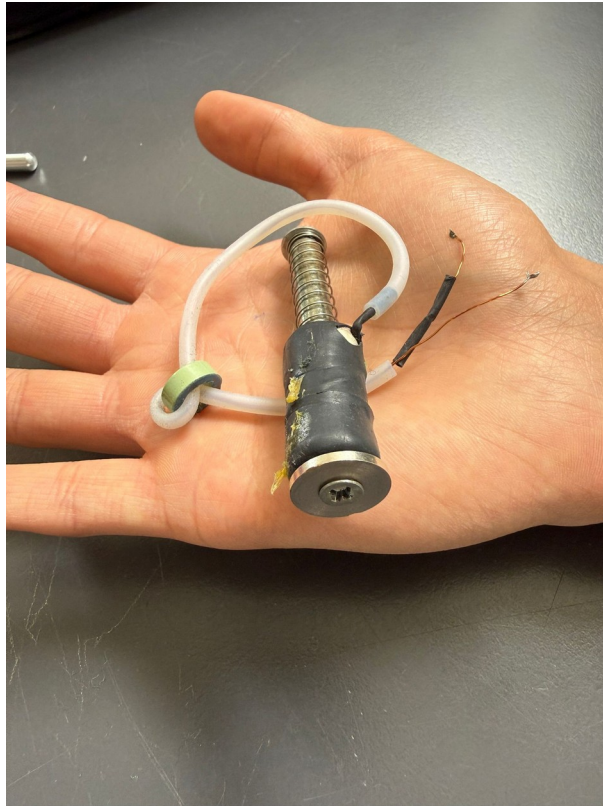
to improve the robot's grip we designed curved suspensions, thus exploiting the elasticity of the lower polycarbonate frame. In order to protect the wheels, we also designed bumpers, as we can see in the image below.



In the bottom view of the robot, you can see the bumpers and the arrangement of the line sensors.

4.3. Kicker

The Kicker was made with a homemade coil, the piston is made half in iron and half in no magnetic material, the iron part is positioned external and kept in pose by a spring. When current flows through the coil, the iron portion is pushed inside the coil, so that the piston can kick the ball. The force of the piston is due basically by two factors: current and magnetic field. In order to increase the power of the kicker we decided to reduce the resistance of the coil, from 0.7Ω . So when the MosFet of the kicker module is driven On by the **MCU**, the burst current is: $\frac{48V}{0.7\Omega} = 69 A$. This means that we have to use an appropriate MosFet.



Our homemade coil. We used a 0.4mm diameter copper wire

4.4. mirror

Vision is the main feature that the robot must have, so we spent a lot of time designing the mirror, trying different profiles. To simulate their behavior, we created a virtual field in the software blender, used to simulate the different mirror profiles.

4.5. Omnidirectional movement

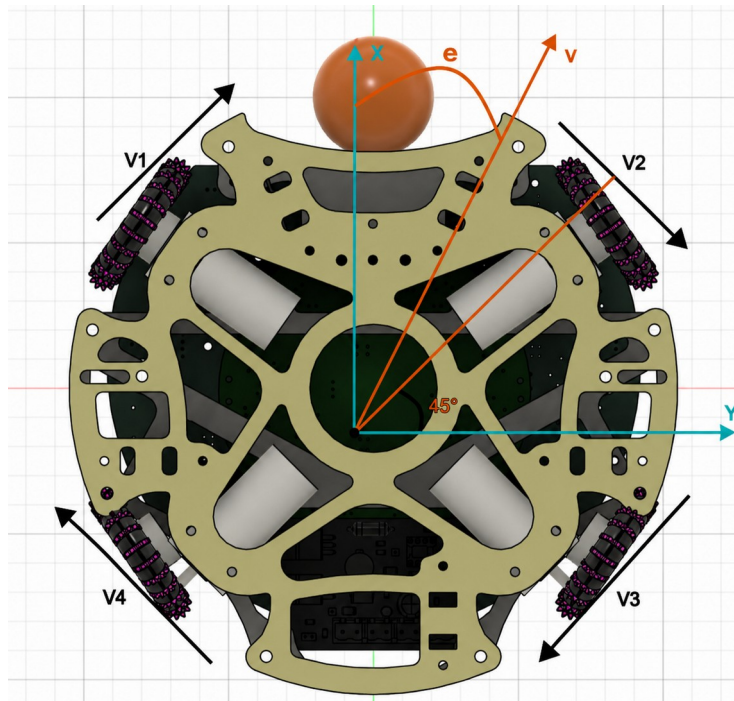
The omnidirectional movement is obtained by composing the four speed vectors of the motors. The robot features an **asymmetric wheel layout**:

wheels V1 and V2 form an angle of **55°** with the X-axis, while wheels V3 and V4 form an angle of **45°** with the X-axis.

The trigonometric values used are:

$$\sin(55^\circ) \approx 0.8192 \quad \cos(55^\circ) \approx 0.5736$$

$$\sin(45^\circ) \approx 0.7071 \quad \cos(45^\circ) \approx 0.7071$$



By decomposing the four vectors along the two axes, we obtain the following **forward kinematics** equations:

$$V_x = (V_1 \cdot \cos 55^\circ - V_2 \cdot \cos 55^\circ - V_3 \cdot \cos 45^\circ + V_4 \cdot \cos 45^\circ)$$

$$V_y = (V_1 \cdot \sin 55^\circ + V_2 \cdot \sin 55^\circ - V_3 \cdot \sin 45^\circ - V_4 \cdot \sin 45^\circ)$$

These equations alone are not directly invertible (since there are 4 unknowns).

To obtain the inverse kinematics, we introduce the no-rotation constraint ($\omega = 0$):

$$V_1 + V_2 + V_3 + V_4 = 0 \quad V_1 + V_2 + V_3 + V_4 = 0 \quad V_1 + V_2 + V_3 + V_4 = 0$$

Once again the equations aren't solvable since the system has 4 unknowns. To achieve the inverse kinematics it's needed to use the matrix calculation. At this point, assuming that the resulting velocity V (composed of V_x and V_y) forms an angle α with the X-axis, we can write:

$$V_x = V \cdot \cos \alpha \quad V_y = V \cdot \sin \alpha$$

Using matrix calculations, the equations for each motor become:

$$V_1 = V \cdot (\cos \alpha \cdot \cos 55^\circ + \sin \alpha \cdot \sin 55^\circ) = V \cdot \cos(\alpha - 55^\circ)$$

$$V_2 = V \cdot (-\cos \alpha \cdot \cos 55^\circ + \sin \alpha \cdot \sin 55^\circ) = V \cdot \cos(\alpha - 125^\circ)$$

$$V_3 = V \cdot (-\cos \alpha \cdot \cos 45^\circ - \sin \alpha \cdot \sin 45^\circ) = V \cdot \cos(\alpha - 225^\circ)$$

$$V_4 = V \cdot (\cos \alpha \cdot \cos 45^\circ - \sin \alpha \cdot \sin 45^\circ) = V \cdot \cos(\alpha - 315^\circ)$$

The speed of each motor is directly proportional to the PWM signal applied to it.

5. Software

Viper and Phantom robots have multiple programs that have to be managed by our team. Since our robots use four different processing units, each written in a different development software and with different

programming languages, we want to split the explanation by processing units, rather than by strategies or competence areas.

5.1. Camera

Due to its mounting position, the PixyCam provides a frontal field of view. Although it typically faces only one goal at a time, the color signatures for both goals are stored in its memory; using the PixyMon software, we can train the camera to recognize multiple color signatures. The camera communicates with the main microcontroller via the I2C protocol, transmitting data on which signature is detected and the size of the recognized blob. Through the control logic in the main code, this real-time data allows us to evaluate whether we have a clear shot, need to adjust our alignment to center the goal, or if an opponent is obstructing the line of sight. Furthermore, a hardware switch on the PCB allows us to quickly select the target goal color before the match.

The OpenMV H7 is mounted facing the top mirror, to give a full 360 view of the field.

Below there is the initialization of the OpenMV H7 and import of all required MicroPython libraries. Camera parameters such as exposure, gain, contrast, and saturation are manually configured and optimized using the OpenMV IDE.

- Library import
- Camera initialization
- Exposure and gain setup
- Image optimization

```
1 import sensor
2 import time
3 import pyb
4 from math import sqrt, atan2, pi
```

```
6 # ===== PARAMETRI CAM
7 sensor.reset()
8 sensor.set_pixformat(sensor.RGB565)
9 sensor.set_framesize(sensor.QVGA)
0 sensor.__write_reg(0x6B, 0x22)
1 sensor.set_vflip(0)
2 sensor.set_hmirror(1)
3 sensor.skip_frames(time=1000)
4
5 sensor.set_auto_whitebal(False)
6 sensor.set_auto_exposure(True)
7 sensor.set_auto_gain(18)
8
9 sensor.set_contrast(5)
0 sensor.set_saturation(0)
1 sensor.set_brightness(0)
2 sensor.skip_frames(time=500)
3
4 clock = time.clock()
```

Thresholds and System Parameters

Definition of all constants and thresholds used for object detection. Color ranges are manually calibrated for ball and goal recognition.

- Ball and goal thresholds
- PI and timing constants
- Direction variables
- LED bootstrap procedure

```
# ===== COSTANTI =====
SOGLIE_COLORE = [
    (16, 92, -105, 88, -46, -13),
    (16, 92, -105, 88, -46, -13),
    (21, 89, -14, -1, 30, 127)
]

SOGLIE_PORTA = [SOGLIE_COLORE[1], SOGLIE_COLORE[2]]

CENTROROBOTX = 140
CENTROROBOTY = 113

PORTA_PERSA = 500
PI_180 = 180 / pi

# ===== VARIABILI =====
direzionePalla = 0
distanzaPalla = 0
tempoUltimaPalla = pyb.millis()

direzionePortaAttacco = 0
distanzaPortaAttacco = 0
tempoUltimaPortaAttacco = pyb.millis()

direzionePortaDifesa = 0
distanzaPortaDifesa = 0
tempoUltimaPortaDifesa = pyb.millis()

# ===== UART INIZIALIZZAZIONE COMUNICAZIONE SERIALE(POLLING) =====
uart = pyb.UART(1)
uart.init(115200, bits=8, parity=None, stop=1, timeout=3, timeout_char=3)

# ===== LED =====
redLed = pyb.LED(1)
greenLed = pyb.LED(2)
blueLed = pyb.LED(3)

# ===== SEQUENZA DI AVVIO LED =====

def avvio_tricolore():
    redLed.on()
    time.sleep(1)
    redLed.off()
    time.sleep(1)
    greenLed.on()
    time.sleep(1)
    greenLed.off()
    blueLed.on()
    time.sleep(1)
    blueLed.off()
```

Image Processing and Detection

Each frame is scanned for color blobs matching the selected thresholds.
Blob filtering removes invalid detections based on size and shape.

- Blob detection
- Ball tracking
- Goal detection
- Distance calculation
- Direction estimation

```
# ===== FUNZIONI =====  
  
def distanzaCm(d):  
    val = 0.1048 * d * d - 13.01 * d + 433.52  
    return max(0, min(255, int(val)))  
  
def trova_palla(macchie):  
    if not macchie:  
        return False, None  
    macchie = [m for m in macchie if m.area() > 10]  
    if not macchie:  
        return False, None  
    macchie.sort(key=lambda x: x.area(), reverse=True)  
    return True, macchie[0]  
  
def trova_porta(macchie):  
    if not macchie:  
        return False, None  
    macchie.sort(key=lambda x: x.area(), reverse=True)  
    return True, macchie[0]  
  
def calcola_distanza_ed_direzione(macchia):  
    x = macchia.cx()  
    y = macchia.cy()  
  
    xr = x - CENTROROBOTX  
    yr = CENTROROBOTY - y  
  
    if xr == 0:  
        xr = 1  
  
    distanza = sqrt(xr * xr + yr * yr)  
    direzione = int((atan2(yr, xr) * PI_180) - 180) * -1  
  
    return direzione, distanza
```

Communications to pic

- UART initialization
- Data packet formatting
- Serial transmission

```
def trasmissioneDati():
    uart.writechar(ord('c'))

    timeout = 0
    start = time.ticks_ms()

    while uart.any() == 0 and timeout < 100:
        timeout = time.ticks_ms() - start

    if uart.any() and uart.readchar() == ord('c'):
        uart.writechar(direzionePalla)
        uart.writechar(direzionePalla >> 8)
        uart.writechar(distanzaPalla)
        uart.writechar(direzionePortaDifesa)
        uart.writechar(distanzaPortaDifesa)
        uart.writechar(direzionePortaAttacco)
        uart.writechar(distanzaPortaAttacco)

    if DEBUG >= 1:
        print("UART > Comunicazione inviata:",
              direzionePalla,
              distanzaPalla,
              direzionePortaDifesa,
              distanzaPortaDifesa,
              direzionePortaAttacco,
              distanzaPortaAttacco)
```

Main Loop

Continuous execution loop handling frame acquisition, object detection, calculations, debug visualization, Transmission of processed coordinates and navigation data to the PIC main board for robot control.

- Frame acquisition
- Blob validation
- Function execution
- Debug drawing
- Real-time processing

```
# ===== PARTE PRINCIPALE DEL PRPGRAMMA =====
avvio_tricolore()

while uart.any():
    uart.readchar()

while True:

    clock.tick()

    img = sensor.snapshot()

    # ===== MASCHERA ANTEPRIMA CAMERA =====
    cx = int(sensor.width() / 2 - 21)
    cy = int(sensor.height() / 2 + 15)

    if DEBUG >= 2:
        print("valore di cx", cx)
        print("valore di cy", cy)

    img.mask_ellipse(cx, cy, 100, 100, 0)
    img.mask_circle(cx, cy, 110)

    # ===== FIRMA PALLA =====
    macchie = img.find_blobs([SOGLIE_COLORE[0]], pixels_threshold=20, area_threshold=20, merge=False)
    # ===== FIRMA PALLA =====
    macchie = img.find_blobs([SOGLIE_COLORE[0]], pixels_threshold=20, area_threshold=20, merge=False)

    for macchia in macchie:
        if macchia.code() == 1:
            # sovraimpressione firma palla
            img.draw_rectangle(macchia.rect(), color=(0, 255, 0))

    trovato, macchia = trova_palla(macchie)

    if trovato:
        blueLed.on()

        nuova_direzione, nuova_distanza = calcola_distanza_ed_direzione(macchia)

        direzionePalla = int(direzionePalla * 0.7 + nuova_direzione * 0.3)
        distanzaPalla = int(distanzaPalla * 0.7 + nuova_distanza * 0.3)

        tempoUltimaPalla = pyb.millis()

    else:
        blueLed.off()

        delta = pyb.millis() - tempoUltimaPalla

        if delta < 1000 and distanzaPalla < 70:
            distanzaPalla = 0
        else:
            distanzaPalla = 0
```

```
# ===== FIRMA PORTE =====
macchie_porte = img.find_blobs(SOGLIE_PORTA, pixels_threshold=75, area_threshold=50, merge=False)

attacco = []
difesa = []

if macchie_porte:
    for macchia_porta in macchie_porte:
        if macchia_porta.code() == 1:
            attacco.append(macchia_porta)
            # sovraimpressione firma porta di attacco
            img.draw_rectangle(macchia_porta.rect(), color=(0, 255, 0))
        elif macchia_porta.code() == 2:
            difesa.append(macchia_porta)
            # sovraimpressione firma porta difesa
            img.draw_rectangle(macchia_porta.rect(), color=(0, 255, 0))

# FIRMA PORTA ATTACCO
trovato, macchia = trova_porta(attacco)

if trovato:
    nuova_direzione, nuova_distanza = calcola_distanza_ed_direzione(macchia)
    direzionePortaAttacco = int(nuova_direzione * 0.708)
    distanzaPortaAttacco = int(distanzaCm(nuova_distanza))
    tempoUltimaPortaAttacco = pyb.millis()

# FIRMA PORTA DIFESA
trovato, macchia = trova_porta(difesa)

# FIRMA PORTA DIFESA
trovato, macchia = trova_porta(difesa)

if trovato:
    nuova_direzione, nuova_distanza = calcola_distanza_ed_direzione(macchia)
    direzionePortaDifesa = int(nuova_direzione * 0.708)
    distanzaPortaDifesa = int(distanzaCm(nuova_direzione))
    tempoUltimaPortaDifesa = pyb.millis()

# ===== UART COMUNICAZIONE SERIALE (POLLING) =====
trasmissioneDati()

# ===== DEBUG FPS =====
if DEBUG >= 3:
    print("FPS:", clock.fps())
```

5.2. Main MCU

5.2.1. Communications management

The main MCU is responsible for handling data received from multiple devices using different communication protocols.

If the ball or the goals are not detected, their respective values are set to “-1”, which serves as a “not found” flag during the decision-making process.

The second highest priority communication is with the secondary MCU. This triggers a low-priority interrupt that temporarily suspends the main program. Inside the interrupt, the main controller reads and stores the current status of the line sensors. Once this is completed, the interrupt routine ends, but it sets a flag in the main control loop. This flag activates the “reentry” condition, which determines the appropriate maneuver to prevent the robot from going out of bounds.

```
// --- CASI COMBINATI, PRIORITA' ALTA ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && LIN_DX_TEMP == 1) // AV e DX
{calcolaPwmDir(sudovest,ERR); goto MANOVR;}

if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && LIN_SX_TEMP == 1) // AV e SX
{calcolaPwmDir(sudest,ERR); goto MANOVR;}

if ((LIN_IND_TEMP == 1 && LIN_DX_TEMP == 1) // IND e DX
{calcolaPwmDir(nordovest,ERR); goto MANOVR;}

if ((LIN_IND_TEMP == 1 && LIN_SX_TEMP == 1) // IND e SX
{calcolaPwmDir(nordest,ERR); goto MANOVR;}

if ((LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULREAR <= 40)
{calcolaPwmDir(avanti,ERR); goto MANOVR;}

if ((LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULFRONT <= 40)
{calcolaPwmDir(indietro,ERR); goto MANOVR;}

// --- CASI SINGOLI ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && FLAG_DIR != 7)
{calcolaPwmDir(indietro,ERR); goto MANOVR;}

if ((LIN_IND_TEMP == 1 && FLAG_DIR != 0 && FLAG_DIR != 1 && FLAG_DIR != 2 && FLAG_DIR != 12 && FLAG_DIR != 13)
{calcolaPwmDir(avanti,ERR); goto MANOVR;}

if ((LIN_SX_TEMP == 1 && FLAG_DIR != 5 && FLAG_DIR != 4)
{calcolaPwmDir(dx,ERR); goto MANOVR;}

if ((LIN_DX_TEMP == 1 && FLAG_DIR != 9 && FLAG_DIR != 10)
{calcolaPwmDir(sx,ERR); goto MANOVR;}

// --- CORREZIONE LATERALE CON ULTRASUONI ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULDX < SOGLIA_PAR && ULSX > SOGLIA_CAMPO)
{calcolaPwmDir(sx,ERR); goto MANOVR;}

if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULSX < SOGLIA_PAR && ULDX > SOGLIA_CAMPO)
{calcolaPwmDir(dx,ERR); goto MANOVR;}

// Fallback finale
switch(lastDir)
{
case 0: case 1: case 2: case 12: case 13:
calcolaPwmDir(indietro,ERR); break;
case 3: calcolaPwmDir(sudovest,ERR); break;
case 4: case 5: calcolaPwmDir(sx,ERR); break;
case 6: calcolaPwmDir(nordovest,ERR); break;
case 7: calcolaPwmDir(avanti,ERR); break;
case 8: calcolaPwmDir(nordest,ERR); break;
case 9: case 10: calcolaPwmDir(dx,ERR); break;
case 11: calcolaPwmDir(sudest,ERR); break;
}
```

This algorithm calculates the angle at which the robot crossed the white line by analyzing the known positions of the line sensors. When one or more sensors are triggered, the system evaluates the active sensor combinations and chooses the most suitable escape maneuver.

In situations where multiple sensors are activated at the same time, priority is given to specific combined patterns (for example, front + side sensors). If no clear combined condition is identified, the system defaults to the opposite direction of the last executed maneuver (lastDir) to bring the robot back inside the field.

The ultrasonic sensors are read directly from the analog pins of the MCU. To minimize noise and ensure more stable readings, each sensor is sampled several times and the arithmetic mean is computed. The resulting filtered values (ULDX, ULSX, ULFRONT, ULREAR) are then used to adjust the escape strategy and avoid obstacles while performing the recovery maneuver.

```
unsigned int ULDX_MED=0;
for (U = 0; U < 3; U++)
{
    // lettura ADC
    ULDX_MED = ULDX + ULDX_MED;
}
ULDX = ULDX_MED / 3;    // ← Media semplice su 3 campioni

unsigned int ULDXv[ULSIZE]={30,30,30}, ...;
unsigned int indexUL=0;
```

5.2.2 Ball following algorithm

To ensure the ball-tracking algorithm performed reliably, we defined a precise set of directional movements for the robot. Given the high

accuracy demanded by the control logic — both for locating the ball and avoiding obstacles — our approach to achieving the smoothest and most accurate motion relies on individually calculated PWM values for each motor and direction. The robot supports a total of 14 different movement directions.

To determine these values, we created a custom Excel spreadsheet that, using an offset and a normalization factor as inputs, generates the appropriate PWM values for all four motors.

Fattore di normalizzazioe	350	offset	0	Offset fisso 130
---------------------------	-----	--------	---	------------------

		pwm5 SUL PIC18F66K80			
		pwm1	pwm2	pwm3	pwm4
	Max				
0	0,70	302	350	-302	-350
15	0,88	312	188	-152	-349
30	1,00	320	75	-46	-349
45	1,05	326	-19	42	-349
60	0,98	350	-117	134	-367
90	0,78	350	-350	350	-350
135	1,05	42	-349	326	-19
180	0,70	-302	-350	302	350
225	1,05	-326	19	-42	349
270	0,78	-350	350	-350	350
300	1,03	-127	349	-332	111
315	1,05	-42	349	-326	19
330	1,03	45	339	-310	-73
345	0,88	152	349	-312	-188

PWMs calculation workflow in Excel

```
// === LOGICA SEGUI PALLA ===
if(DIR == 0) {
    // NESSUNA PALLA RILEVATA
    NOPALLA++;
    LED_PWR = 1;

    if(NOPALLA >= 100) {
        // Cerca palla girando
        RUOTA_SX();
        Delayms(100);
        // Verifica se ha trovato palla
        LEGGI_IR();
        if(DIR == 0) {
            RUOTA_DX();
            Delayms(200);
        }
    }
}
else {
    // PALLA RILEVATA
    NOPALLA = 0;
    LED_PWR = 0;

    // Scegli direzione in base alla posizione della palla
    switch(DIR) {
        case 5: // PALLA AVANTI
            AVANTI();
            break;

        case 3: // PALLA SINISTRA
        case 4: // SINISTRA DIAGONALE
            SX();
            break;

        case 7: // PALLA DESTRA
        case 6: // DESTRA DIAGONALE
            DX();
            break;

        case 13: // PALLA DIETRO
            INDIETRO();
            break;

        default:
            AVANTI();
            break;
    }
}

// === CONTROLLO OSTACOLI ULTRASUONI ===
if(ULDX < SOGLIA_SICU) {
    // Ostacolo a destra -> gira a sinistra
    SX();
    Delayms(50);
}

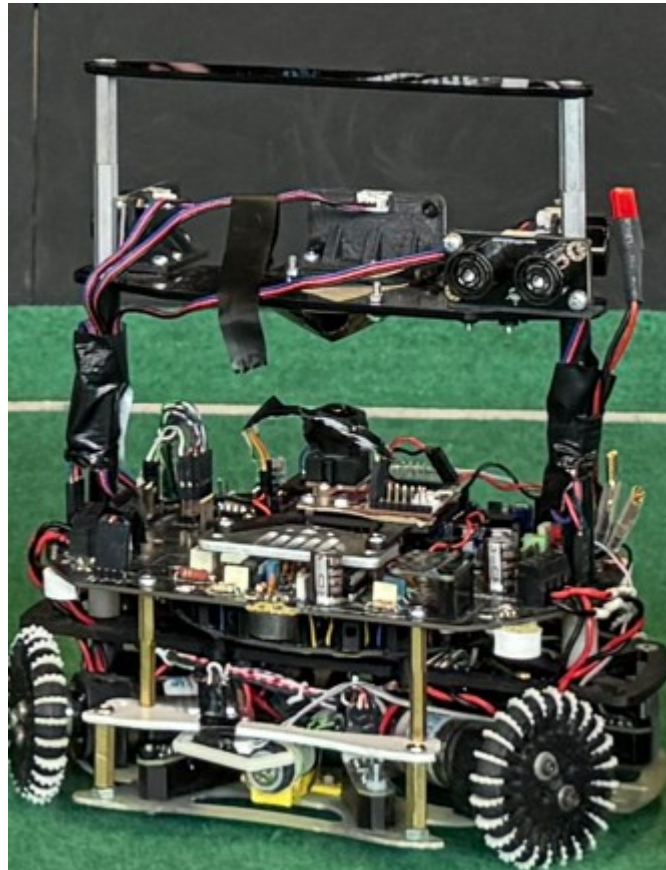
if(ULSX < SOGLIA_SICU) {
    // Ostacolo a sinistra -> gira a destra
    DX();
    Delayms(50);
}

// === PICCOLA PAUSA PER STABILITA' ===
Delayms(20);
}
```

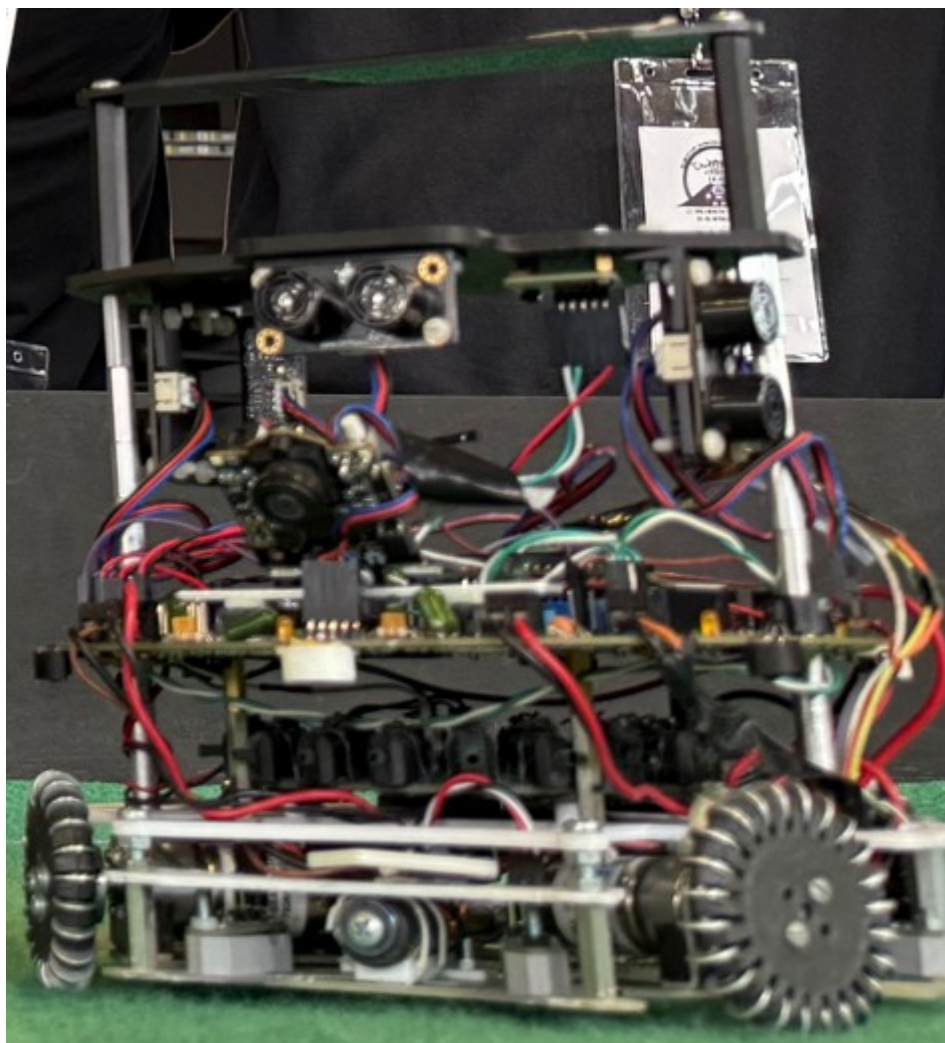
5.3. Secondary MCU

The secondary MCU is responsible for reading the 10 analog line sensors. Its firmware is written entirely in assembly language. This controller also manages an OLED display, which is used for debugging and tuning purposes. The processor includes a calibration algorithm that automatically generates the activation thresholds for the lines. This is done after calibrating the sensors two at a time by placing them in specific conditions prior to the start of the match.

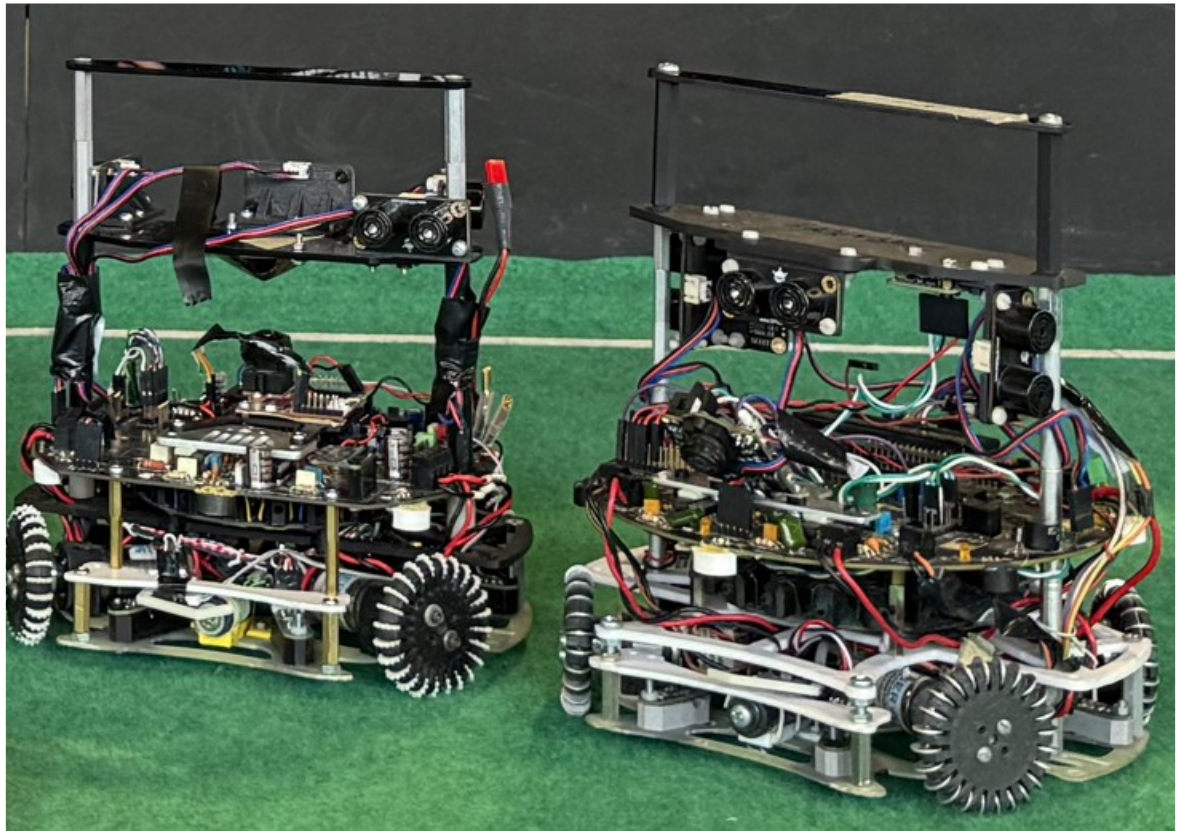
6. GALLERY



Robot Viper S³



Robot Phantom



Viper S³ and Phantom on the field at the Italian tournament